

Einschätzungsfragen

Im Rahmen der Challenge interessieren mich Deine Einschätzungen zu den folgenden Fragen:

Tech-Stack: Welchen Stack erachtest du im CRM/ERP Kontext am Sinnvollsten und wieso?

Meine Empfehlung:

- **Backend:** Python (FastAPI) — schnelle Entwicklung, Async-Support, gute OpenTelemetry/metrics-Ökosystem, viele ORMs (SQLModel/SQLAlchemy).
- **DB / Persistenz:** PostgreSQL (für Prod & Dev-Envs). Wobei ich hier auch externe Dienstleister wie z.B. Xano in Betracht ziehen würde.
- **Frontend:** Flutter ist aus meiner Sicht die beste Wahl, weil es eine einheitliche, leistungsstarke und zukunftssichere Plattform bietet, die Desktop, Web und Mobile abdeckt und die Mitarbeiter nicht auf ein Gerät, sondern auf ihre Aufgaben fokussiert. Es zwingt zu klarer, modularer UX und reduziert Komplexität, während es gleichzeitig Flexibilität und Skalierbarkeit für die Arbeit überall ermöglicht.
- **Auth & Infra:** JWT / OAuth2 (Keycloak/Auth0 optional). Docker Compose für lokale Runs; Deploy: Render/Vercel + Managed Postgres oder Kubernetes für Scale.
- **Observability:** Prometheus + Grafana (metrics), OpenTelemetry traces, strukturierte JSON-Logs (e.g. via loguru/structlog).
- **Background-Jobs:** Redis + RQ / Celery / Dramatiq (wenn nötig).
Warum: Schnell aufzusetzen, sehr gutes developer experience, leicht zu skalieren, große Community & Bibliotheken, passende für CRM/ERP-Integrationen (webhooks, queues, batch jobs).
Trade-offs: Python + FastAPI ist sehr productive but single-process defaults require proper worker setup (uvicorn/gunicorn). Postgres hat höheren operational overhead vs SQLite.

End-to-End Interface: Welche Variante (UI, API oder CLI) hast du gewählt und warum? Skizziere den Flow aus Sicht eines Sales-Users.

End-to-End Interface — gewählte Variante + Sales-Flow:

Variante: UI (Flutter) + JSON REST API (FastAPI).

Warum: Sales-User profitieren von interaktiver Oberfläche (Schnelligkeit, visuelle Feedbacks). API bleibt für Integrationen/automation.

Flow (Sales-User, Beispiel: Lead → Opportunity → Quote):

1. Sales logs in (JWT).
2. Auf Dashboard: „Neuen Lead anlegen“ (Formular). Frontend sendet POST `/api/v1/leads` mit `tenant_id` (aus Token).
3. Lead erscheint in Liste (GET `/api/v1/leads?tenant_id=...`). React Query cached und aktualisiert UI.

4. Sales klickt auf Lead → „Convert to Opportunity“ (POST `/api/v1/opportunities`), UI zeigt optimistic update (Opportunity erscheint sofort).
5. Background job (optional) validiert E-Mail / enrich via external API; Ergebnis wird per WebSocket/ServerSentEvent an UI gepusht und persisted.
6. Sales erstellt Quote, speichert, und kann Status/History sehen (audit trail).

Minimale API Endpoints hier:

- POST `/api/v1/leads`
- GET `/api/v1/leads`
- POST `/api/v1/leads/{id}/convert` → opportunity
- GET `/api/v1/opportunities`
- POST `/api/v1/quotes`

Observability: Welche strukturierten Logs/Metriken/Traces baust du ein? Was würdest du im Alerting überwachen?

Logs (strukturiert JSON):

- Common fields: `timestamp`, `level`, `service`, `env`, `tenant_id`, `request_id`, `user_id`, `path`, `method`, `status`, `duration_ms`, `msg`.

Metriken (Prometheus):

- HTTP request count by route/status/method.
- Latency histograms (p50/p95/p99) per endpoint.
- Error rates (5xx / 4xx).
- DB stats: connection usage, query durations, slow queries.
- Job queue metrics: queue length, job failures, job processing time.
- Auth failures & rate limit events.
Traces: OpenTelemetry across frontend→backend→DB→external APIs to see end-to-end latency and hot paths.
Alerting (examples & thresholds):
- Error rate (5xx) > 1% over 5m → Pager.
- p95 latency for critical endpoints > 1s (or product-defined SLO) → Pager.
- DB connections > 80% of pool → Pager.
- Queue length rising for > 10m or job failure rate > 5% → Ops.
- Repeated auth failures for tenant (>100 failed logins / 10m) → security alert.

Frontend (falls UI): Wie organisierst du State/Fetching (z. B. React Query). Fehler-/LoadingZustände und ggf. Optimistic Updates?

State & Fetching:

- React Query (TanStack) als zentrale Fetch/Caching-Schicht: automatisch stale-while-revalidate, retries, background refetch, cache invalidation.
- Global UI state (Auth, Theme) via Context + Zustand/MobX only if necessary; keep React Query for server state.
Fehler / Loading Zustände:
- Komponenten zeigen skeletons / spinners bei initial load.
- Inline errors per-form → toast (non-blocking) + inline field errors.
- Global error boundary (React Error Boundary) für unrecoverable UI errors with option to reload.
Optimistic Updates:

- Bei Aktionen wie „Lead anlegen“ oder „Status ändern“: apply optimistic update to cache (React Query `useMutation` with `onMutate`), rollback on failure.
- Use server timestamps / request_id to reconcile race conditions.

Retries / Idempotency:

- Mutations are idempotent where possible (idempotency-key header). For long running ops, show pending state and later reconcile via polling or websockets.

Performance & Skalierung: Wo liegt im Prototyp der Engpass und wie würdest du ihn heben?

Wahrscheinlicher Engpass (Prototyp):

- DB (Postgres): single instance, slow queries, missing indexes, connection exhaustion.
- Single node API process (uvicorn) — blocking long tasks.
Wie beheben:
 - Kurzfristig (low effort):
 - Add indexes for query patterns (tenant_id + created_at).
 - Use DB connection pooling (pgbouncer) and tune pool sizes.
 - Move heavy tasks to background worker (Redis + RQ/Celery).
 - Add HTTP caching / CDN for static content.
 - Mittelfristig:
 - Read replicas for heavy read workloads; split reads/writes.
 - Add Redis cache for hot entities (session, commonly read lists).

- Containerize & autoscale API pods behind load balancer (K8s/HPA).
- Long-term / Architecture:
 - Shard/partition tenants if massive scale (range/horizontal).
 - Use CQRS for write-heavy workflows (separate write path + read materialized views).
 - Aggregate metrics and sampling traces to reduce overhead.

Produktnutzen: Welche Sales-Persona profitiert konkret und wie würdest du Impact messen?

Konkrete Personas:

- Account Executive (AE): braucht schnellen Überblick zu Leads/Opportunities, History, und schnelles Erstellen von Quotes → spart Zeit bei Konvertierungen.
 - Sales Development Rep (SDR): arbeitet Leads täglich; profitiert von Enrichment, deduplizierung, und schneller Lead-Creation.
 - Sales Ops / Manager: braucht Reports, Conversion Rates, Pipeline Velocity.
- Wie Impact messen (KPIs):
- Adoption: DAU/MAU (Anzahl Sales-User, die Feature nutzen).
 - Time saved: durchschnittliche Zeit zur Lead-Erstellung / Conversion (Vorher vs Nachher).
 - Conversion metrics: Lead→Opportunity rate, Opportunity→Closed-Win rate.
 - Pipeline Velocity: average time in stage (goal: reduce).
 - Revenue impact: uplift in win rate or faster closes → ARR uplift (harder to attribute but critical).
 - Operational metrics: reduction in manual followups, fewer duplicate leads.

Next Steps: Was würdest du mit +1 weiterem Tag bauen und was bewusst nicht?

Mit +1 Arbeitstag (realistisch & maximaler Product/Dev Value):

1. Auth + Multi-tenant enforcement (JWT + middleware verifying `tenant_id` against token).
2. Minimal API + UI flow: Lead list, Create Lead form, Convert to Opportunity. (React + FastAPI endpoints).
3. Persistence: Postgres schema + basic migration (alembic) or SQLite for demo. DB model includes `tenant_id` and indices.
4. Observability quick wins: structured logs + basic Prometheus metrics endpoint (`/metrics`).
5. Docker Compose to run API + DB + frontend locally.
6. README: architecture sketch, setup/run, trade-offs, next steps.

Was bewusst NICHT bauen in +1 Tag:

- Vollständiges RBAC/permissions-system beyond basic tenant isolation.
- Complex data enrichment integrations/third-party connectors.
- Full analytics dashboards & reports (Grafana setup beyond a sample dashboard).
- Advanced multi-tenant isolation like separate schemas per tenant (no time to implement & operational cost).
- Full enterprise features: audit-compliance exports, legal/security hardening beyond basics.

```
[Flutter UI] <---> [FastAPI (uvicorn, gunicorn workers)] <---> [Postgres]
```

```
| -> /metrics -> Prometheus -> Grafana
```

```
| -> traces -> OpenTelemetry collector
```

- | -> Redis (cache / job queue)

Background Workers <-